

AutoClock: Automated Clock Management for Power-Efficient HLS Designs on FPGAs

Jiawei Liang^{1,2}, Linfeng Du¹, Xiaofeng Zhou¹, Zhe Lin³, Jiang Xu⁴, Wei Zhang^{1,2,*}

¹The Hong Kong University of Science and Technology, ²Guangzhou HKUST Fox Ying Tung Research Institute

³Sun Yat-sen University, ⁴The Hong Kong University of Science and Technology (GZ)

{jliangbr, linfeng.du, xzhoubu}@connect.ust.hk, linzh235@mail.sysu.edu.cn, jiang.xu@hkust-gz.edu.cn, wei.zhang@ust.hk

Abstract—High-level synthesis (HLS) tools streamline FPGA design by enabling engineers to implement hardware using C/C++ languages. However, while clock management serves as a critical stage in the FPGA EDA flow that affects system-level performance, area, and especially power consumption, existing commercial HLS tools lack comprehensive solutions for clock management. Specifically, the diversity of clock resources creates a vast design space for finding the optimal configuration, and the insufficient analysis of multiple clock domain scenarios hinders effective clock-oriented optimizations in HLS. This work introduces AutoClock, an open-source integrated clock management framework that complements AMD Vitis HLS. AutoClock allocates resources for clock generation, assigns modules to appropriate clock domains, addresses metastability and time division multiplexing (TDM) malfunctioning introduced by multi-clock domain architectures, and hierarchically gates the clock of modules in a design. Experimental results demonstrate that AutoClock can fully utilize clock resources on FPGAs and help reduce dynamic power consumption by up to 74.38%.

Index Terms—High-level synthesis, clock management, clock domain crossing, clock gating, clock multiplexer.

I. INTRODUCTION

Field-programmable gate array (FPGA) is widely used in various fields, such as autonomous driving [1]–[3], data center [4]–[6], and deep learning algorithm acceleration [7]–[9], owing to its reconfigurable nature with low latency. As applications become increasingly complex, the requisite time for their implementation on FPGA has increased exponentially. To address this problem, *high-level synthesis (HLS)* has gained significant traction as a methodology that enables engineers with limited hardware design expertise to use high-level programming languages for application implementation, largely bypassing the demand for learning specific low-level hardware minutiae.

Despite the popularity of HLS for FPGA, few commercial or academic tools have placed a focus on clock resource management, although the clock is one of the most essential components of an FPGA design. On the one hand, the use of clock resources is highly related to *performance, power, and area (PPA)* metrics of FPGA designs, particularly the power consumption [10], [11]. On the other hand, when serving as an *application-specific integrated circuit (ASIC)* prototyping platform, FPGA should maintain the same clock behaviors as on ASIC. The inherent disparities in clock resources between FPGAs and ASICs have historically necessitated manual reconstruction of clock topology for FPGA-based prototypes,

which involves a time-consuming process. Consequently, empowering HLS to manage clock resources automatically would not only mitigate the power dissipation of FPGA designs but also enable HLS to support more complex applications while accelerating the overall development cycle for engineers.

Clock resources available in AMD FPGAs include clock buffers such as BUFGMUX and BUFGCE, as well as clock management tiles containing *phase-locked loop (PLL)* and *mixed-mode clock manager (MMCM)* [12]. Though previous works have explored various approaches for managing clock resources in FPGA designs, their efforts addressed only a limited spectrum of obstacles. Some challenges remain when seeking to fully leverage these resources from the HLS level:

- Existing studies have largely focused on PLLs for clock generation [13] or conceptualized digital clock management modules without delving into the practicalities of resource selection [10]. However, modern FPGAs, such as those from AMD, offer a diverse array of clock resources with similar functionalities but distinct power profiles. This diversity, coupled with the finite availability of these resources, presents a considerable challenge in optimizing power consumption while meeting design specifications.
- Besides, prior works have predominantly address specific *clock domain crossing (CDC)* scenarios, such as memory sharing across two clock domains and handshake mechanisms [13], or double buffering [14]. Various types of multi-clock domain designs present a challenge in automatically handling all types of CDC cases appropriately.
- Additionally, the limited number of clock gates in FPGAs demands a strategic allocation methodology. Nevertheless, existing HLS-based clock gating insertion work [15] has primarily dealt with small benchmarks and has not addressed the limitation of resources.
- Finally, while previous research [15] has explored clock gating in streaming modules using buffer status for back-pressure, real-world FPGA applications often encompass a broader range of modules, necessitating more versatile clock gating methods that can accommodate the diverse module interactions.

To address these challenges, this paper proposes the first integrated, automatic clock resource management framework, named AutoClock, aimed at presenting power optimization of HLS designs on the AMD Vitis flow. AutoClock leverages

* Corresponding author.

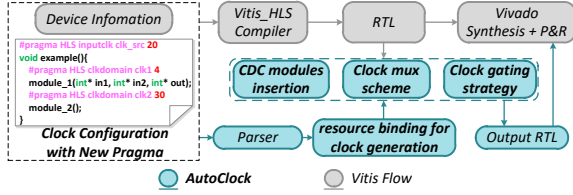


Fig. 1. Overview architecture of AutoClock.

clock buffers and clock management tiles while considering a broader range of CDC and clock gating scenarios. Additionally, the quantity and power consumption of these resources are considered to minimize overall power cost. Our contributions are highlighted as follows:

- We develop an open-source clock resource management framework that addresses a significant gap in current HLS tools. The open-source repository is available at: <https://github.com/MasLiang/AutoClock>.
- By formulating the resource binding of clock generation as an *integer linear programming (ILP)* problem featuring iterative constraint refinement, and coupling it with a greedy hierarchical clock gating strategy, AutoClock minimizes the overall power consumption of FPGA designs.
- AutoClock provides adaptive automatic CDC modules insertion for various interfaces and signals, as well as a clock multiplexer-based scheme to handle diverse multiple clock domain scenarios.

II. AUTOCLOCK FRAMEWORK DESIGN

The overall flow of the proposed clock management framework is illustrated in Fig. 1. Firstly, a pair of new pragmas are introduced that allow users to configure clock domains within an HLS design, accompanied by a dedicated parser to process these pragmas. Based on the parsing result, AutoClock generates a *clock reset generator (CRG)* for clock generation and reset synchronization while binding modules and related signals to different clock domains. Subsequently, AutoClock inserts clock gates into various types of modules in each clock domain using a greedy, hierarchical strategy to significantly minimize dynamic power consumption. Afterward, issues introduced by multiple clock domains are solved by inserting appropriate CDC modules automatically for various types of signals, including async-FIFO, async-BRAM, level- and pulse-synchronizer, to ensure reliable data transfer between clock domains, and applying clock multiplexer scheme to handle TDM modules invoked by multiple clock domains. Finally, the modified RTL code with clock management is fed into Vivado for further implementation.

A. Clock Configuration with New Pragmas

For compatible with Vitis HLS with supporting multiple clock domains configuration, a pair of new custom pragmas is proposed as listed in Table I. With these pragmas, users can easily configure the clock domain and target frequency of each HLS function. The parser processes these pragmas, using naming conventions to associate configurations with

TABLE I
AUTOCLOCK-SPECIFIC PRAGMAS: TEMPLATES AND DESCRIPTIONS.

Coding Template	Description
<code>#pragma HLS inputclk <name> <frequency></code>	Used before the top module to define clock frequency of the input clock.
<code>#pragma HLS clkdomain <name> <frequency></code>	Used before a module to define the clock domain of this module.

corresponding RTL modules. The pragmas are then removed from the project to ensure compatibility with Vitis HLS, while the extracted clock domain information is used in subsequent steps of the AutoClock framework.

B. Resource Binding for Clock generation

Given the clock frequencies specified in the user configuration, multiple types of clock resources can be leveraged for generating them, namely BUFGCE_DIV and clock management tiles, including MMCM and PLL, in which MMCM can support the most complex functionality while BUFGCE_DIV costs the least power. Our purpose is to find the optimal binding of these resources that can satisfy the design requirements and consume minimum power. The problem is formulated as:

Problem. Given the input system clock frequency f_{in} , the list of clock frequencies needing to generate $\mathbf{F} = \{f_i\} \in \mathbb{R}$, and the set of available clock resources $\mathbf{S}$. The goal of resource binding for clock generation is to identify a set of clock resources S_f that fulfill $\mathbf{F} = g(f_{in}; \mathbf{S}_f)$ while minimizing the power consumption under the constraints of resource quantity and design performance.

Given the large number of clock resources and broad exploration space, the problem is formulated as an ILP problem featuring iterative constraint refinement for speedup. To simplify the problem, we first traverse all the frequency values to find the ones that BUFGCE_DIV can generate under the constraint of resource number. Subsequently, the remaining part of the problem can be solved as an ILP problem:

$$\begin{aligned}
 & \min(n_{PLL} \times p_{PLL} + n_{MMCM} \times p_{MMCM}) \\
 & s.t. \ n_{PLL} \leq N_{PLL} \\
 & \quad n_{MMCM} \leq N_{MMCM} \\
 & \quad N f_{PLL}^{max} \times n_{PLL} + N f_{MMCM}^{max} \times n_{MMCM} \geq n_{freq} \\
 & \quad n_{PLL} \in \mathbb{Z} \\
 & \quad n_{MMCM} \in \mathbb{Z}
 \end{aligned} \tag{1}$$

where n_s and p_s mean the number and power consumption of clock resource of type s , respectively, N_s is the upper limit of each type of resource, $N f_s^{max}$ represents the maximum number of frequency that a type of resource can generate. After obtaining the solution, the feasibility of it is validated by considering additional physical constraints. Frequencies in $\mathbf{F}$ are grouped into subsets based on the solution above, namely $\{\mathbf{F}_i^{MMCM}, \mathbf{F}_j^{PLL}\}$ representing clock frequencies bound to an MMCM or a PLL. Each subset must satisfy the requirement of its corresponding resource. Take a subset of clock frequency

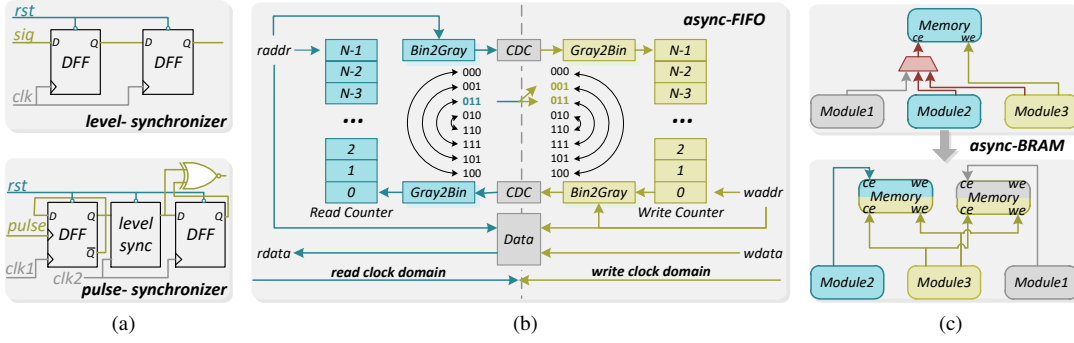


Fig. 2. CDC modules. (a) single-bit synchronizers for level- and pulse- signals. (b) async-FIFO. (c) async-BRAM.

bound to an MMCM as an example. MMCM contains a global multiplier, a global divider, and some specific dividers for each output clock. Input frequency is processed by the global multiplier and divider to produce an intermediate value, which is then divided by specific dividers to generate output frequencies. Therefore, there are certain constraints associated with the MMCM:

$$\begin{aligned}
 &\exists \alpha, \beta \in \mathbb{R} \\
 &s.t. \frac{\alpha}{f_{in}} \in \mathbf{factor}_{mult_g} \\
 &\quad \frac{\alpha}{\beta} \in \mathbf{factor}_{div_g} \\
 &\quad \frac{\beta}{f_k} \in \mathbf{factor}_{div_k}
 \end{aligned} \quad (2)$$

in which α and β represent results of the global multiplier and divider, $\mathbf{factor}_{mult_g}$, $\mathbf{factor}_{div_g}$, and $\mathbf{factor}_{div_k}$ denotes the valid factors of the global multiplier, the global divider, and divider for k -th output clock in an MMCM, respectively. PLL has a similar constraint. It should be noted that there are many combinations when grouping these clock frequencies, and a solution is feasible as long as there is a combination that meets the requirements. If a solution is infeasible, a new constraint is introduced to exclude it from the solution space:

$$s.t. n_{PLL} \times p_{PLL} + n_{MMCM} \times p_{MMCM} > P_{solution} \quad (3)$$

where the $P_{solution}$ is the power consumption of the failed solution. Then the ILP solver will be exerted again with the new constraint. This process is iterated until a feasible solution is found.

C. CDC Modules Insertion

Since different clock domains have varying features, such as frequency and phase, signals crossing clock domains must be synchronized to prevent metastable states from being sampled and propagating, which leads to incorrect circuit behavior. To satisfy this requirement, a series of CDC modules, including single-bit synchronizers, namely synchronizers for pulse- and level-signals, as well as multi-bit synchronizers, i.e., *asynchronous first-in first-out (async-FIFO)* and *asynchronous block random access memory (async-BRAM)*, are tailored to deal with a different type of signals.

1) *Single-bit synchronizer*: There are two types of single-bit signals: level signals and pulse signals, between which level signals are the most common in digital circuits. The top part of Fig. 2a shows a two-flip-flop synchronizer for single-bit level signals. Since it is used as a part of the handshake mechanism in [13], there is no need to care about the relationship between the frequency of two clock domains. However, this synchronizer has limitations when being applied independently. As illustrated in Fig. 3a, the width of the signal in the faster clock domain might be narrower than the period of the slower clock domain, leading to the risk of missing sampling. To mitigate this risk, AutoClock expands those level signals crossing the clock domain as Fig. 3b so that they can always be properly sampled in the slower clock domain.

Pulse signals are commonly used in control circuits. When this type of signal crosses clock domains, the two-flip-flop synchronizer can not fulfill the requirement. For instance, a pulse from the slower clock domain that sustains longer than twice the period of the faster clock domain will be sampled twice, generating a level signal after synchronizing. Therefore, a dedicated pulse-synchronizer is employed, shown in the bottom part of Fig. 2a. In this synchronizer, the input pulse serves as an enable signal to control the conversion of a level signal, which will cross the clock domain boundary relying on the level-synchronizer. In the destination clock domain, the edge of the synchronized level signal triggers the generation of a pulse signal. Fig. 3d gives the timing diagram for this process. Due to the constraint of level-synchronizer, the width of the level signal, i.e., the interval between two pulses T_{pulse} , must exceed the period of the target clock.

In addition to regular signals, the reset signal is another significantly important type of signal. Similar to regular signals needing to satisfy setup and hold time constraints, reset signals must satisfy the requirement of the recovery and removal time constraint as Fig. 3c, or metastable state might propagate. As a result, reset signals must also be synchronized for each clock domain when de-assertion. The reset-synchronizer shares the same module with the level-synchronizer depicted in the top part of Fig. 2a with the input signal set as constant 0.

2) *multi-bit synchronizer*: CDC for multi-bit signals introduces more complexity. Although the reliability of two-flip-flop based level-signal synchronizer has been proven in [13]

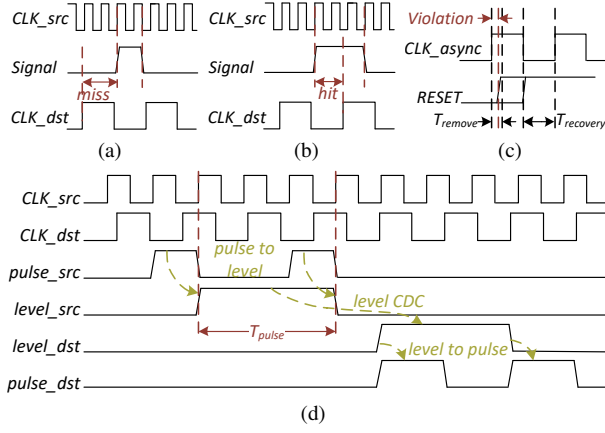


Fig. 3. Timing analysis of synchronizers. (a) Miss of signal sampling. (b) After expanding, the signal is sampled by the slower clock. (c) A violation example of removal time. (d) The timing of the pulse synchronizer.

to have an enormous *mean time before failure (MTBF)*, the MTBF of n parallel level-synchronizers for multi-bit signals will decrease by n times [16]. To address this problem, async-FIFO and async-BRAM are applied. While there are some other types of CDC modules, such as synchronizers based on handshake mechanism [16], they can be viewed as variations of previous basic synchronizers and will not be analyzed in detail here.

Async-FIFO is used when sequential multi-bit signal transfers. As illustrated in Fig. 2b, an async-FIFO includes shared data storage space and two address pointers binding to two clock domains. Pointers are converted to Gray Code before being synchronized because only a single bit changes in Gray Code when the address value increases. In this way, the MTBF is equal to a single-bit synchronizer. Moreover, there are only two conditions when Gray Code crosses clock domains: 1) successfully sampling and 2) failing to sample the changed bit that the value remains unchanged. For instance, in Fig. 2b, when the original r -address changes from 001 to 011, the synchronized r -address will either change to 011 successfully or remain at 001. If it remains, the state of the r -pointer in the write clock domain does not change. Thus, async-FIFO can guarantee the correctness of reading and writing.

BRAM is widely used as a shared data storage space among modules for transferring data. When a BRAM is shared by two clock domains, two BRAM ports are bound to different clock domains [13]. However, BRAM might be scheduled with time division multiplexing to be accessed by multiple modules for minimizing resource cost, e.g., as shown in the top part of Fig. 2c, *module3* writes data to the BRAM while *module1, 2* from different clock domains read data via a multiplexer. In this case, the BRAM will be copied for each read clock domain and memory accessing via multiplexer is separated to multiple independent accessing.

D. Clock multiplexer scheme

To limit resource allocation in the implemented design and reduce resource utilization, Vitit-HLS supports TDM for

Pseudo-Algorithm 1. Hierarchical greedy clock gating strategy

Input: the number of BUFGCEs g_n
the number of level L
the number of modules at l -th level N_l
the number of gated module $N_g = 0$
input design with a set of modules $M = \{m_0^0, \dots, m_{n-1}^{L-1}\}$

Output: Set of gated module M_g

```

1: for  $l = 0$  to  $L$  do
2:   for  $n = 0$  to  $N_l$  do
3:     calculate power that can be saved using Equation. 6
4:     sort modules by saved power
5:     for  $n = 0$  to  $N_l$  do
6:       if father module is dataflow module do
7:         skip this module
8:       else do
9:         insert BUFGCE for clock gating
10:         $N_g += 1$ 
11:        if  $N_g == g_n$  do
12:          break
13:        if  $N_g == g_n$  do
14:          break
15:    if  $N_g == g_n$  do
16:      break

```

modules. However, multi-clock domains strategy makes it malfunctioning because a module is confined to a single clock domain. Thus, invoking the same module in multiple clock domains requires creating copies for each clock domain. It will introduce additional resource utilization as well as static power cost. To address this issue, a clock multiplexer scheme is proposed. For modules utilized across multiple clock domains, a new clock domain is created that incorporates multiple clocks with different frequencies corresponding to clock domains initiating these modules. In the protocol from Xilinx official document [17], an *finite state machine (FSM)* is implemented to control status of each kernel. A selection signal is then generated based on the FSM states to indicate which clock domain is invoking the module. With this signal, a BUFGMUX is instantiated to select the specific clock frequency. By implementing this scheme, the module can still function as a TDM module. Additionally, to mitigate potential signal instability because of frequency conversion, redundant idle states are inserted to the FSM. This remains sufficient time for the clock multiplexing process to ensure stable operation.

E. Clock Gating Strategy

Dynamic power dissipation, as described by Eq. 4, is closely related to active ratio. When signals remain, clock-related transistors still guzzle energy since their active ratio is 100%, including logic elements like LUT and DFF, as well as buffers and programmable switches on the route of the clock tree. To save such unnecessary power consumption, BUFGCEs are inserted into specific modules as clock gates in the clock tree to disable clock signals when their associated functions are idle. Due to the limited number of BUFGCEs on a given FPGA, a hierarchical greedy clock gating strategy, as illustrated in Pseudo-Algorithm 1, is advised for efficiently binding these BUFGCEs to maximize power saving.

$$\begin{aligned}
P_{total} &= P_{static} + P_{dynamic} \\
&= Vdd \times I_{leakage} + \sum_{w \in W} \alpha_w C_w Vdd^2 f \quad (4)
\end{aligned}$$

Firstly, there are two kinds of modules, i.e., dataflow modules whose sub-modules run continuously in the pipeline and non-dataflow modules where sub-modules exert in sequence. Due to the continuous working of sub-modules in a well-optimized dataflow module throughout the entire module execution, independent clock gating for them is unnecessary. For non-dataflow modules, some of their sub-modules might remain inactive when they are exerting. Thus, clock signals can be gated during the idle of these sub-modules to reduce power consumption. Previous works [11], [15] only care about dataflow modules and formulate them as an actor controlled through FIFO interface for inserting gates. When expanding to non-dataflow modules, the FIFO interface is not enough to cover all cases. Thus, the state of FSM in each module is used. Namely, when the FSM is in idle state, the clock is disabled.

Based on Xilinx documents [17], there are three block-level control protocols: *ap_ctrl_chain*, *ap_ctrl_hs*, and *ap_ctrl_none*. To generate an FSM for a block, *ap_ctrl_none* protocol is prohibited from use. In other protocols, the two signals, *ap_start* and *ap_done*, indicate the states of the FSM, thus are used to assert and de-assert the enable signals of clock gates. It is worth noting that the clock driving the FSM should always be on. Otherwise, deadlock will occur: if the clock is disabled, the state of the FSM will never change, and the clock will not be enabled. Additionally, the synchronous reset strategy is widely used in FPGA design, so the clock should be enabled when synchronous reset signals are asserted.

Besides, when a BUFGCE is inserted into a module, it influences circuits within this module and its sub-modules. Consequently, gating higher-level modules can save power more effectively than gating lower-level modules. Based on this, clock gates are inserted into modules from higher to lower levels. At the same level, all modules are sorted by power saving, which is closely related to two factors. First, the amount of saved power is linearly related to gated time. Second, modules with more transistors will result in more power saving in the same number of gated cycles. In a conclusion, the power that can be saved in a module is represented by:

$$P_{saved} = \frac{C_{idle}}{C_{total}} \times \sum_{t \in T} P_t \quad (5)$$

Where C is the number of cycles extracted from Vitis HLS reports, T is the set of basic elements in FPGA related to clock signals, including buffers in clock tree, LUTs, registers, DSP, and BRAM, and P_t is the power of an electronic element t in a cycle, obtained from AMD Xilinx Power Estimator. Since the idle time of a module overlaps with its higher-level modules, the clock gating time will also be calculated repeatedly. As a result, those time slots that have already been gated by clock gates in higher-level modules should be excluded when comparing with other modules. Thus, the Eq. 5 becomes:

$$P_{saved} = \frac{C_{idle}^l - \sum_{k=0}^{l-1} C_{idle}^k}{C_{total}} \times \sum_{t \in T} P_t \quad (6)$$

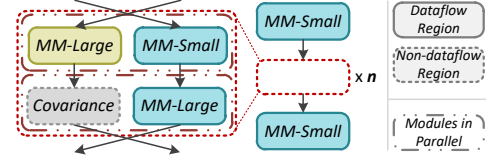


Fig. 4. Topology of benchmarks PnMC. Different background color represent different clock domains.

in which C_{idle}^l and C_{idle}^k are the number of idle cycles of module at level l and its higher level modules, respectively. Subsequently, modules are sorted by the amount of saved power, and clock gates are allocated to those modules that have greater power savings.

III. EXPERIMENT

A. Experimental Setup and Benchmarks

The proposed AutoClock is implemented in Python with the Pyverilog tool [18] to parse and generate RTL code. All experiments are conducted on the AMD Alveo U280 FPGA based on Vitis 2023.2. *Xilinx Run Time (XRT)* is applied to measure power consumption, with power counter readings taken at *1ms* intervals. Each benchmark is executed 10,000 times successively to obtain a reasonable average power cost. Each experiment is repeated 20 times to eliminate random errors. For normalization, the system clock frequency of all benchmarks is set to *100MHz*.

To effectively evaluate AutoClock, a series of benchmarks are designed. Since the Vitis flow lacks clock management features, conventional HLS design benchmark suites such as Vitis Libraries [19] and Rosetta [20] are developed with a single clock domain, thus not clearly presenting the advantages of AutoClock. Similar to [21], we generate two dataflow kernels using AutoSA [22], namely *MM-Large* and *MM-Small* with the same matrix multiplication function but different latency, and a non-dataflow kernel *Covariance* from PolyBench [23]. We combine these to create composite large designs for generality in non-dataflow protocol. As these kernels have different latencies, they can be bound to different clock domains when working in parallel. Fig. 4 illustrates the structure of benchmarks used in this section, where PnMC represents cascading n parallel blocks. The background colors represent the clock domains to which these kernels belong. In the parallel block, two pairs of kernels work in parallel for multi-clock domain strategy analysis. They are also designed to demonstrate the effectiveness of the clock multiplexer scheme, as *MM-Large* works across different clock domains.

B. Clock Gating Evaluation

To thoroughly evaluate the impact of clock gating, we limited the number of BUFGCEs to 10 to address corner cases. Table II presents the comparison results. As benchmarks expand from P1MC to P3MC, the power reduction ratio consistently remains high, demonstrating the effectiveness of the clock gating method regardless of design scale. Additionally, since static power consumption is clock-independent, we focus on dynamic power. The results indicate that dynamic

TABLE II
POWER EVALUATION WITH MULTI-CLOCK DOMAINS STRATEGY AND CLOCK GATING ON BENCHMARKS WITH PARALLEL BLOCKS.

Benchmark	Total power (W)	Dynamic power (W)	Resource		
			LUT	FF	BRAM
P1MC	1.58	0.55	256235 (19.65%)	335103 (12.85%)	906.5 (44.97%)
P2MC	1.53	0.88	341516 (26.20%)	445657 (17.09%)	1326 (65.77%)
P3MC	1.95	1.31	426115 (32.69%)	556248 (21.33%)	1745.5 (86.58%)
P1MC_cg	0.87 (-23.92%)	0.27 (-50.20%)	257203 (+0.08%)	339255 (+0.16%)	906.5 (+0.00%)
P2MC_cg	1.01 (-34.12%)	0.36 (-59.57%)	341867 (+0.02%)	450863 (+0.20%)	1326 (+0.00%)
P3MC_cg	1.13 (-41.90%)	0.49 (-62.35%)	426601 (+0.03%)	563456 (+0.28%)	1745.5 (+0.00%)
P3MC_cg_rand1	2.04 (+4.77%)	1.40 (+7.11%)	426457 (+0.03%)	556226 (+0.00%)	1745.5 (+0.00%)
P3MC_cg_rand2	1.93 (-1.09%)	1.29 (-1.63%)	426248 (+0.01%)	556261 (+0.00%)	1745.5 (+0.00%)
P3MC_cg_rand3	1.95 (+0.28%)	1.31 (+0.41%)	426230 (+0.00%)	556265 (+0.00%)	1745.5 (+0.00%)
P3MC_cg_dfs	1.70 (-12.84%)	1.06 (-19.10%)	426546 (+0.03%)	561953 (+0.22%)	1745.5 (+0.00%)
P3MC_cg_bfs	1.14 (-41.29%)	0.50 (-61.44%)	426409 (+0.02%)	561411 (+0.20%)	1745.5 (+0.00%)
P1MC_cdc	1.09 (-4.93%)	0.54 (-9.48%)	255976 (-0.02%)	340237 (+0.20%)	922 (+0.76%)
P2MC_cdc	1.29 (-15.45%)	0.73 (-24.45%)	341534 (+0.00%)	452899 (+0.26%)	1357 (+1.54%)
P3MC_cdc	1.58 (-19.14%)	1.03 (-26.64%)	426159 (+0.00%)	565563 (+0.36%)	1792 (+2.31%)
P1MC_cdc_cg	0.76 (-33.84%)	0.21 (-65.12%)	257297 (+0.09%)	340268 (+0.20%)	922 (+0.76%)
P2MC_cdc_cg	0.81 (-47.01%)	0.25 (-74.38%)	341640 (+0.01%)	452915 (+0.28%)	1357 (+1.54%)
P3MC_cdc_cg	0.97 (-50.41%)	0.42 (-70.16%)	426533 (+0.03%)	565599 (+0.36%)	1792 (+2.31%)

power consumption can be reduced by up to 62.35%, similar to results from manually inserting clock gates at the RTL level [24], and greater than HLS-based work [15], which achieved a 53.7% reduction and focused solely on small dataflow designs.

Furthermore, to demonstrate the advantage of the proposed BUFGCE allocation strategy, random selection, *breadth-first search (BFS)*, and *depth-first search (DFS)* strategies were applied to P3MC to allocate a limited number of BUFGCEs. For random selection, conducted three times with different seeds, power cost increases in corner cases because the additional logic from clock gating consumes more power than it saves. The results demonstrate that our strategy achieves more significant power cost reduction. Resource utilization is also reported; although enabling signals introduces extra logic, the increases in LUT and FF usage are minimal.

C. Multi-Clock Domains Strategy Evaluation

The advantages of the multi-clock domains strategy are shown in Table II. In the baseline design, all modules within a parallel block run at the same clock frequency but exhibit varying latencies, causing faster modules to idle while waiting for slower ones. In such cases, modules with lower latency can be assigned a lower clock frequency. For example, the latency of *MM-Large* is 1/4 of *MM-Small*, corresponding to a clock frequency of $100\text{MHz} \times 1/4$. As recorded in Table II, total and dynamic power can be reduced by up to 19.14% and 26.64%, respectively. Additionally, since BRAMs are used between kernels, extra BRAMs are required for CDC. However, only 2.31% of BRAMs for CDC are used in the corner case P3MC, which is acceptable. Given that BRAM is more likely to be the bottleneck than LUT, our benchmarks consist of numerical kernels, so async-FIFO and async-BRAM are widely used for CDC, helping explore the upper limit of BRAM usage. In designs with less data transfer and more control signals, the additional BRAM cost will be lower, while LUT consumption will be higher.

TABLE III
POWER AND AREA EVALUATION FOR CLOCK MULTIPLEXER SCHEME.

Flow	Total power(W)	Dynamic power(W)	LUT	DFF	BRAM
Vitis	0.75	0.29	230326	305926	762.5
AutoClock	0.67	0.21	230451	306992	799.5

We report the power consumption for these benchmarks using both methods. Clock gating proves to be more effective in saving power by disabling clock signals, while the multi-clock domain strategy only lowers frequency, even when modules are idle. These two methods are relatively independent and can be used simultaneously for greater power savings. In our benchmarks, dynamic power savings of up to 74.38% are achieved. Notably, although P3MC shows higher absolute dynamic power reduction, P2MC outperforms P3MC in power reduction ratio due to having fewer kernels and lower dynamic power. Subsequently, the TDM optimization strategy is applied to P1MC, limiting the *MM-Large* kernel in the RTL design to one instance. As shown in Table III, AutoClock still achieves a 25.58% reduction in power dissipation without compromising the effectiveness of the TDM optimization strategy and maintaining resource utilization.

IV. CONCLUSION

This paper introduces AutoClock, an open-source automatic clock management framework for HLS designs using the Vitis flow. AutoClock assists engineers in efficiently utilizing FPGA clock resources and reducing the power consumption of HLS designs. Our experiments demonstrate the effectiveness of AutoClock, achieving dynamic power savings of up to 74.38%.

V. ACKNOWLEDGMENT

This research is supported partly by National Key R&D Program of China (2023YFB4405100, 2023YFB4405101), General Research Fund (16214123), the National Natural Science Foundation of China (62404257), and Guangdong Basic and Applied Basic Research Foundation (2024A1515013155).

REFERENCES

- [1] Y. Li, S. E. Li, X. Jia, S. Zeng, and Y. Wang, "Fpga accelerated model predictive control for autonomous driving," *Journal of intelligent and connected vehicles*, vol. 5, no. 2, pp. 63–71, 2022.
- [2] A. Kojima, "Autonomous driving system implemented on robot car using soc fpga," in *2021 International Conference on Field-Programmable Technology (ICFPT)*, pp. 1–4, IEEE, 2021.
- [3] Y. Liu, S. Hao, K. Huang, M. Yang, Z. Huang, X. Qi, Y. Li, L. Zhou, Y. Long, and J. Zhou, "An fpga-based ultra-high performance and scalable optical flow hardware accelerator for autonomous driving," in *2024 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5, IEEE, 2024.
- [4] C. Wang, K. He, R. Fan, X. Wang, W. Wang, and Q. Hao, "Cxl over ethernet: A novel fpga-based memory disaggregation design in data centers," in *2023 IEEE 31st Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pp. 75–82, IEEE, 2023.
- [5] C. Bobda, J. M. Mbongue, P. Chow, M. Ewais, N. Tarafdar, J. C. Vega, K. Eguro, D. Koch, S. Handagala, M. Leeser, *et al.*, "The future of fpga acceleration in datacenters and the cloud," *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, vol. 15, no. 3, pp. 1–42, 2022.
- [6] Z. Wang, H. Huang, J. Zhang, F. Wu, and G. Alonso, "FpgaNIC: An FPGA-based versatile 100gb SmartNIC for GPUs," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, (Carlsbad, CA), pp. 967–986, USENIX Association, July 2022.
- [7] Y. Wong, Z. Dong, and W. Zhang, "Low bitwidth cnn accelerator on fpga using winograd and block floating point arithmetic," in *2021 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, pp. 218–223, 2021.
- [8] S. Abi-Karam, R. Sarkar, D. Xu, Z. Fan, Z. Wang, and C. Hao, "Inr-arch: A dataflow architecture and compiler for arbitrary-order gradient computations in implicit neural representation processing," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pp. 1–9, IEEE, 2023.
- [9] X. Jia, Y. Zhang, G. Liu, X. Yang, T. Zhang, J. Zheng, D. Xu, Z. Liu, M. Liu, X. Yan, *et al.*, "Xvdp: A high-performance cnn accelerator on the versal platform powered by the ai engine," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 17, no. 2, pp. 1–24, 2024.
- [10] G. Lhairech-Lebreton, P. Coussy, and E. Martin, "Hierarchical and multiple-clock domain high-level synthesis for low-power design on fpga," in *2010 International Conference on Field Programmable Logic and Applications*, pp. 464–468, 2010.
- [11] F. Ratto, T. Fanni, L. Raffo, and C. Sau, "Mutual impact between clock gating and high level synthesis in reconfigurable hardware accelerators," *Electronics*, vol. 10, no. 1, p. 73, 2021.
- [12] I. Advanced Micro Devices, "Ultrascale architecture clocking resources user guide," [Online], 2023.
- [13] O. Ragheb and J. H. Anderson, "High-level synthesis of fpga circuits with multiple clock domains," in *2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pp. 109–116, IEEE, 2018.
- [14] T. Liang, J. Zhao, L. Feng, S. Sinha, and W. Zhang, "Hi-clockflow: Multi-clock dataflow automation and throughput optimization in high-level synthesis," in *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pp. 1–6, IEEE, 2019.
- [15] E. Bezati, S. Casale-Brunet, M. Mattavelli, and J. W. Janneck, "Clock-gating of streaming applications for energy efficient implementations on fpgas," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 36, no. 4, pp. 699–703, 2017.
- [16] S.-J. Kim, J.-G. Lee, and K. Kim, "A parallel flop synchronizer for bridging asynchronous clock domains," in *Proceedings of 2004 IEEE Asia-Pacific Conference on Advanced System Integrated Circuits*, pp. 184–187, 2004.
- [17] "Vitis high-level synthesis user guide (ug1399)," [Online]. <https://docs.amd.com/r/en-US/ug1399-vitis-hls/pragma-HLS-allocation>.
- [18] "Pyverilog," [Online]. <https://github.com/PyHDI/Pyverilog/>.
- [19] "Xilinx. 2022. vitis accelerated libraries," [Online]. https://github.com/Xilinx/Vitis_Libraries/.
- [20] Y. Zhou, U. Gupta, S. Dai, R. Zhao, N. Srivastava, H. Jin, J. Featherston, Y.-H. Lai, G. Liu, G. A. Velasquez, W. Wang, and Z. Zhang, "Rosetta: A Realistic High-Level Synthesis Benchmark Suite for Software-Programmable FPGAs," *Int'l Symp. on Field-Programmable Gate Arrays (FPGA)*, Feb 2018.
- [21] L. Du, T. Liang, S. Sinha, Z. Xie, and W. Zhang, "Fado: F loorplan-a ware d irective o ptimization for high-level synthesis designs on multi-die fpgas," in *Proceedings of the 2023 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, pp. 15–25, 2023.
- [22] J. Wang, L. Guo, and J. Cong, "Autosa: A polyhedral compiler for high-performance systolic arrays on fpga," in *Proceedings of the 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2021.
- [23] T. Y. Louis-Noel Pouchet, "Polybench/c," [Online], 2016.
- [24] M. H. Vo, "The merged clock gating architecture for low power digital clock application on fpga," in *2018 International Conference on Advanced Technologies for Communications (ATC)*, pp. 282–286, 2018.